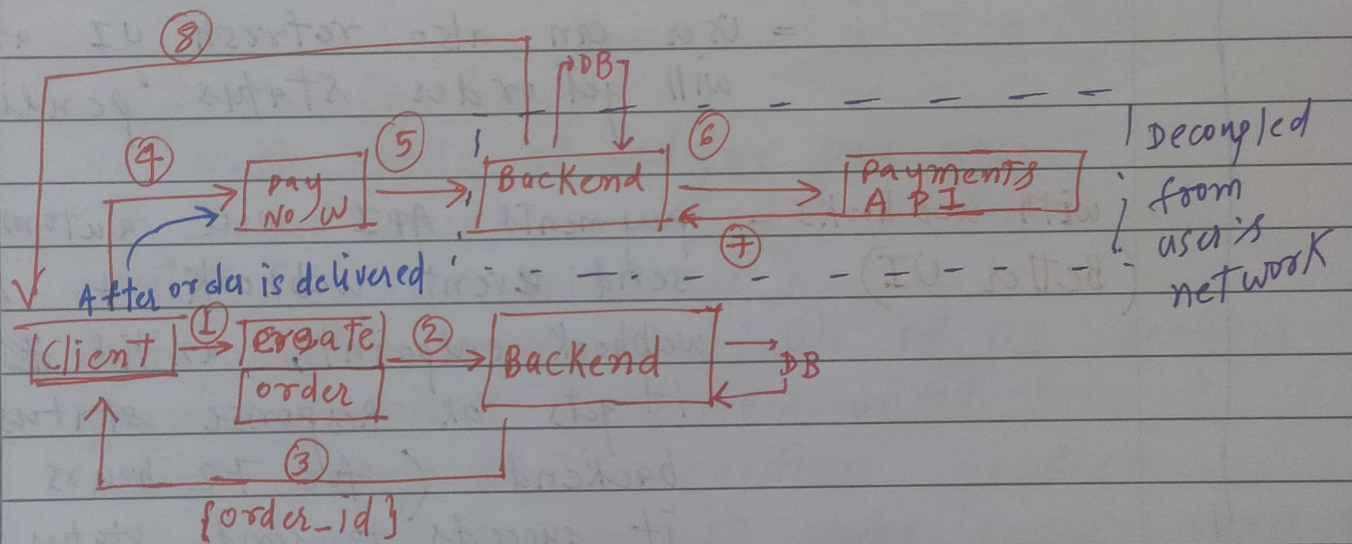




payment failures -

COD Case

Failure point ④ → (a) client network is back in some time → UI shows order ^{progress} pending, payment pending → Refresh → calls order status → Return Response (success or failure) ↓ give Retry option

(DB &) Backend knows the status.

payment processing started → ~~order~~ ^{not created} (depending on pay status)
↳ no response to client

(b) This is kind of relatable for cases when pay first
client network back in long time after 24
hours or even longer → Respond with
status of order → (success or fail)

Failure point (f) (Payments API → Backend)

Without webhooks → payment processed (success)
failure

= Backend would need to keep track of
status, by requesting from payments
API via some scheduled cron jobs.

= User can also refresh UI and
will get order status "pending"

with webhooks - payments API will automatically
(Better UI) send event webhook to backend
webhook endpoint, until ~~it~~
it gets ok response status from
backend. (for 72 hours until
it succeeds to send status to
backend)

Payfirst Database single source of truth

1) ~~order created~~ → order created → order_id generated
2) ~~payments endpoint~~ ^{pay now (pay i key)}
Initiate payment for (order_id, payments i_key)
(pay i key, items_payload) (for current transaction)
payment status → create order
to be deleted after 24 hours.

For any issues with full system failure (network*) for more than 72 hours (Backend will query using cron works to get status of pending payments).

COD
Case

for why use i-key if using order id?

(i-key & order-id combination (ung.)) will be used to validate replay requests.

same (I-Key, order-id) → return cached result with response

different order id, with same I-Key → reject it (possibly fraud)

Robust Frontend

1. No double clicks allowed for pay now. (if clicked once, cannot be clicked again until it gets response)
2. Retry option only available if UI receives payment failure response, until then it shows order & transaction pending.
3. When user refreshes it should query order status endpoint (~~by~~ /order-id/ as order is already

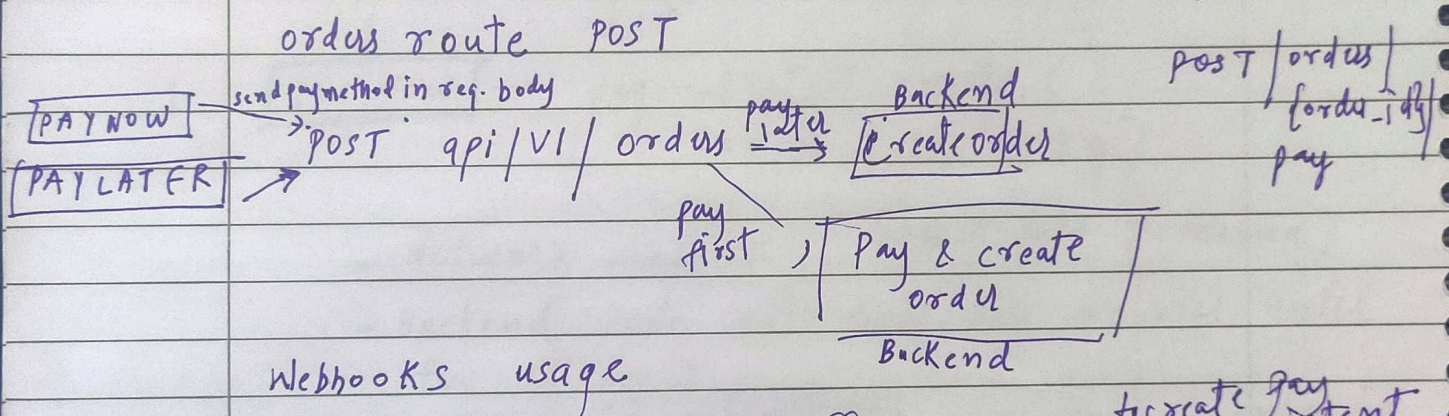
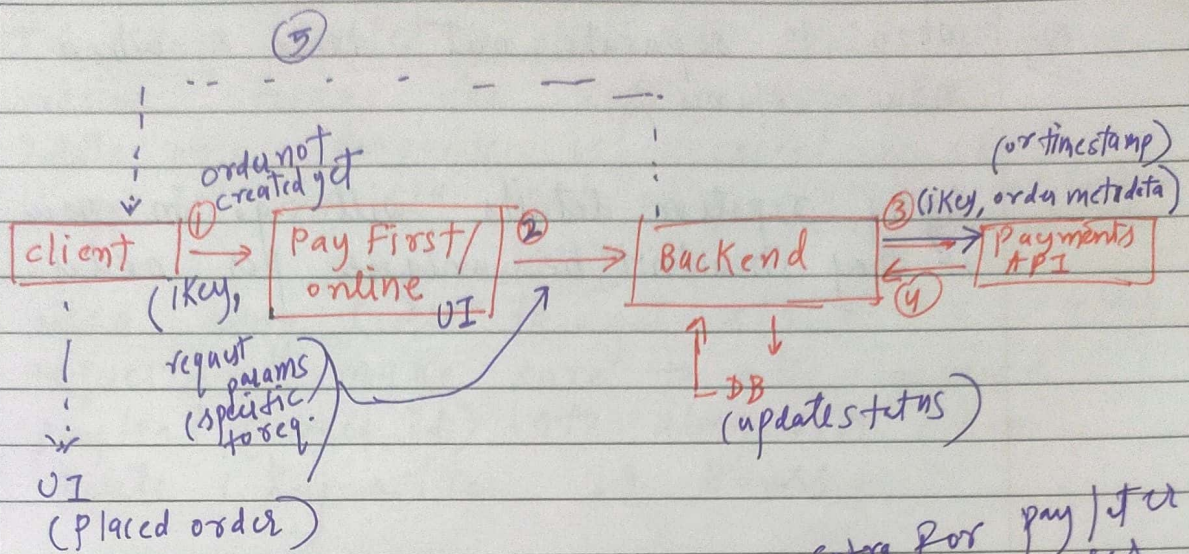
- 1) If IKey generated in client side, retrieve it from session storage for replay requests. Delete once response is received by client. Keep it for 24 hours only. (similarly for backend side generated iKey)
- 2) when using Redis to get status for replay requests, make sure to also include payload (order_id) into along with iKey. Delete iKey after 24 hours.

I-Key ~~main~~ usage (COD case)

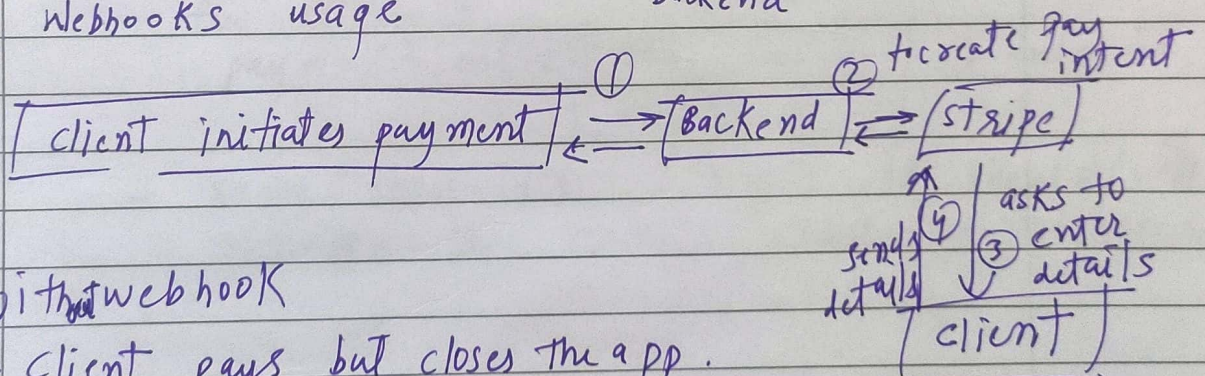
1. pay now → `paymentintent.create( — — — , order.id, i-Key )`
- Network failure (Maybe it was processed)
 - Backend code automatically retries until few attempts
`paymentintent.create( — — — , order.id, i-Key )`
 - same (order.id & i-Key) it should/will return the response that was processed.

2. IKey will be deleted after some time, so same Key cannot be used to initiate fraud transactions over long time.

- payment process will be async so websockets will be used to send response to client.



Webhooks usage



Issues with webhooks

- Client pays but closes the app.
 - Backend doesn't know payment status as client directly sent data to stripe (4)

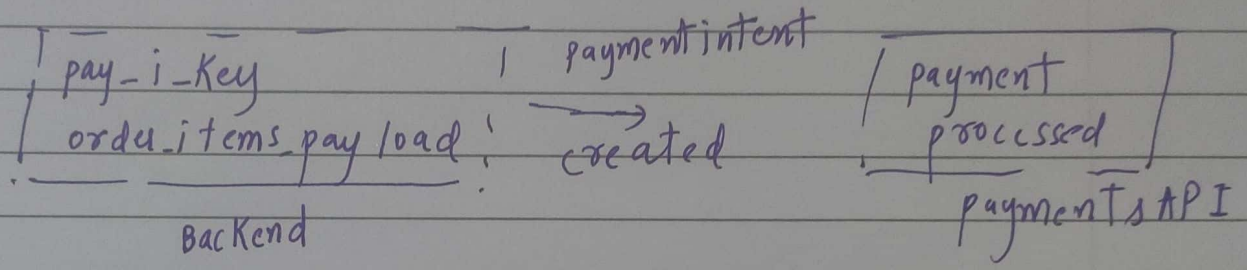
solns.

- Front end queries backend again, so it will check with stripe & update status.
- backend automatically retries by querying calling stripe.

WAY Better - pay API automatically send webhooks when it succeeds/fails (processes)

2. ~~user checks~~ Network issue b/w backend & stripe.

3. Pay. initially succeeds but later bank declines. stripe sends payment intent status.



- = Request arrives to backend (inRedis) store ikey + hash(order's payload) + timestamp
- = Add payment status too after it's avbl.
- = ~~Backend gets the response~~
For any case if request sent again, to identify if it's same request check the stored data ikey, ^{hash}(order payload), payment status